

BioRoofline: Cache-Aware Roofline Analysis of Memory Bottlenecks in Parallel Genomic Sequence Alignment Workloads

SOMAK GOSWAMI, Virginia Polytechnic Institute and State University, USA

WU-CHUN FENG, Virginia Polytechnic Institute and State University, USA

Whole-genome sequencing is a backbone of precision medicine yet its central computational bottleneck, genomic sequence alignment via BWA-MEM, exhibits parallel scaling collapse beyond 8–16 threads due to an uncharacterized memory hierarchy bottleneck. This work presents *BioRoofline*, a cache-aware roofline analysis framework that identifies and quantifies the architectural root cause of this collapse. Using GEM5 architectural simulation of a validated BWT FM-index access pattern workload (10,000 alignment queries, 150 bp read length), we conduct 15 controlled experiments varying L1/L2/L3 cache size, prefetcher type, cache line size, associativity, and DRAM generation. We demonstrate that BWT FM-index traversal is deeply memory-bound, that L3 cache provides zero benefit because the human genome FM-index (5.7 GB) exceeds all cache capacities and that a combined 4 MB L2 with stride prefetching achieves the best result at +45.5% IPC. Reducing cache line size from 64 to 32 bytes improves IPC by 10.5% by eliminating bandwidth waste intrinsic to irregular DNA index traversal, a finding with direct implications for clinical parallel genomic deployments.

CCS Concepts: • **Computer systems organization** → **Parallel architectures**; **Memory hierarchies**; *Multicore architectures*; • **Applied computing** → *Bioinformatics*.

Additional Key Words and Phrases: roofline model, whole-genome sequencing, DNA alignment, cache hierarchy, GEM5 simulation, BWA-MEM, BWT FM-index, memory-bound workloads, parallel scaling, biomedical HPC, arithmetic intensity, prefetching, cache line size, DRAM bandwidth

ACM Reference Format:

Somak Goswami and Wu-Chun Feng. 2026. BioRoofline: Cache-Aware Roofline Analysis of Memory Bottlenecks in Parallel Genomic Sequence Alignment Workloads. 1, 1, Article 1 (May 2026), 10 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

1.1 The Clinical Genomics Pipeline

The human genome encodes approximately 20,000 protein-coding genes across 3.2 billion base pairs of DNA. Whole-genome sequencing (WGS) reads every base of a patient’s genome that enables comprehensive variant detection for clinical diagnosis of inherited diseases, cancer driver mutations and pharmacogenomic responses. A clinical WGS workflow produces hundreds of millions of short reads (150 base pair (bp) fragments) that must be mapped back to the human reference genome. This mapping step is both computationally dominant and clinically critical: misaligned reads produce false variant calls that could lead to incorrect diagnoses.

The standard clinical pipeline is the GATK Best Practices workflow [6, 8], whose first and most computationally intensive stage is genomic sequence alignment using BWA-MEM [4]. The total pipeline takes 8–14 hours on a modern 32-core server. In neonatal intensive care units, where rapid genomic diagnosis can guide life-saving treatment, this latency is clinically unacceptable [7].

Authors’ Contact Information: Somak Goswami, somakg63@vt.edu, Bradley Department of Electrical and Computer Engineering, Virginia Polytechnic Institute and State University, Blacksburg, Virginia, USA; Wu-Chun Feng, wfeng@vt.edu, Bradley Department of Electrical and Computer Engineering, Virginia Polytechnic Institute and State University, Blacksburg, Virginia, USA.

2026. ACM XXXX-XXXX/2026/5-ART1
<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

, Vol. 1, No. 1, Article 1. Publication date: May 2026.

1.2 The Alignment Bottleneck

BWA-MEM uses the Burrows-Wheeler Transform (BWT) FM-index: a compressed representation of the reference genome enabling efficient substring search. The FM-index for the human reference genome (GRCh38) occupies approximately 5.7 GB in memory. Alignment performs backward search through this index via LF-mapping: for each base in a read, the aligner looks up two non-contiguous positions in the suffix array. Because consecutive DNA bases may map to completely different genomic loci, these accesses appear random to the memory system.

BWA-MEM parallelizes across reads — each of the 600 million reads in a 30× sample aligns independently, making the problem embarrassingly parallel. Yet Heldenbrand et al. [2] document that BWA-MEM throughput saturates beyond 8–16 threads. The root cause has been attributed qualitatively to memory bandwidth saturation, but the specific cache hierarchy mechanism has never been systematically characterized.

1.3 Research Contributions

This work presents *BioRoofline*, applying the Cache-Aware Roofline Model (CARM) [3, 11] and GEM5 simulation [1, 5] to provide a systematic cache hierarchy analysis of BWT-based DNA alignment. Contributions are:

- C1. BioRoofline framework:** A GEM5-based roofline toolkit with a statically-linked BWT FM-index access pattern simulator (`bwt_sim`).
- C2. First systematic cache characterization:** 15 controlled GEM5 experiments varying L1/L2/L3 size, prefetcher type, cache line size, associativity, and DRAM generation.
- C3. Architectural findings:** (i) L3 cache provides zero benefit; (ii) larger L2 reduces miss latency by 36.8%; (iii) stride prefetching reduces miss count by 4.2%; (iv) combined L2+Stride achieves +45.5% IPC; (v) 32-byte cache lines improve IPC by 10.5%.
- C4. Clinical hardware guidance:** Evidence-based recommendations for cache hierarchy design supporting parallel genomic alignment at scale, extending prior BWT acceleration work from our group [12].

2 Background and Related Work

2.1 DNA Sequencing and the Alignment Problem

Modern Illumina sequencers produce paired-end reads: two 150 bp fragments from opposite ends of a DNA molecule. A 30× WGS sample contains approximately 600 million read pairs, representing the full genome at 30-fold coverage. This redundancy is biologically necessary as clinical variant calling requires sufficient depth at each genomic position to distinguish true variants from sequencing errors (typical error rate: 0.1% per base).

Table 1 summarizes how this work differentiates from prior efforts in genomic alignment optimization and roofline analysis.

2.2 The Burrows-Wheeler Transform in DNA Alignment

BWA-MEM [4] aligns reads using the BWT FM-index of the reference genome. The FM-index enables alignment in $O(L)$ time where L is read length, via the LF-mapping operation:

$$\text{LF}(i) = C[c] + \text{Occ}(c, i - 1) \quad (1)$$

where $c = \text{BWT}[i]$, $C[c]$ is the cumulative character count, and $\text{Occ}(c, i)$ counts occurrences of c in the first i BWT characters. Consecutive LF-mapping steps access non-contiguous memory positions determined by the biological content of the DNA read, creating the pointer-chasing behavior that is the focus of this work.

2.3 Parallel Scaling of Clinical Alignment

BWA-MEM parallelizes alignment across reads using POSIX threads. All threads share the same FM-index, creating implicit competition for cache and memory bandwidth. Heldenbrand et al. [2] observed throughput

Table 1. Differentiation from Prior Work

Work	Scope	Gap Addressed Here
Williams et al. [11]	Roofline, HPC kernels	No genomics workload
Ilic et al. [3]	CARM model	Not applied to BWT
Zhang et al. [12]	BWT on multicores	No roofline analysis
Vasimuddin et al. [9]	BWA-MEM2 SIMD	No cache hierarchy model
Heldenbrand et al. [2]	GATK4 thread scaling	No simulation/roofline
Vera et al. [10]	CARM on GPU	GPU only; no DNA alignment

saturation beyond 16 threads. Vasimuddin et al. [9] improved single-thread performance via SIMD vectorization of BWA-MEM2 but did not characterize the cache hierarchy bottleneck. BWA-MEM-SCALE [13] balanced I/O and compute stages. None applied a roofline analysis to the per-thread memory bottleneck.

2.4 The Roofline Model and CARM

The roofline model [11] characterizes performance as a function of arithmetic intensity I (FLOP/byte):

$$\text{Attainable}(I) = \min(P, I \times B) \quad (2)$$

where P is peak FLOP/s and B is peak memory bandwidth (GB/s). The ridge point $I^* = P/B$ separates memory-bound from compute-bound regimes. For the Intel Core Ultra 285H ($P \approx 150$ GFLOP/s, $B \approx 68$ GB/s LPDDR5), $I^* \approx 2.2$ FLOP/byte. CARM [3] adds separate L1, L2, L3, and DRAM bandwidth ceilings.

2.5 GEM5 Architectural Simulation

GEM5 [1, 5] is a cycle-accurate architectural simulator supporting x86, ARM, and RISC-V. Its TimingSimpleCPU model accurately simulates memory hierarchy behavior — cache hits and misses, DRAM latency, and bandwidth utilization.

3 Methodology

3.1 Biological Workload Characterization

The computational core of DNA alignment is the LF-mapping operation. For a 150 bp read, alignment performs 150 LF-mapping steps (one per nucleotide: A, T, G, or C). Each step requires two random accesses into the Occ array. Because consecutive bases in a DNA read may map to completely different genomic loci, consecutive LF-mapping accesses are near-random from the memory system’s perspective. This access pattern has two key properties:

- (1) **No spatial locality across reads:** Different DNA reads align to different genomic regions, accessing entirely different parts of the FM-index.
- (2) **Limited spatial locality within reads:** Within a single read alignment, consecutive steps may access nearby index positions along unique genomic regions, creating some stride regularity at the cache line level.

3.2 BWT FM-Index Access Pattern Simulator

Running real BWA-MEM inside GEM5 requires a statically-linked binary, which BWA-MEM's build system does not support. We developed `bwt_sim`: a BWT access pattern simulator that models the two-access-per-step memory pattern of LF-mapping while being statically compilable for GEM5 SE mode.

`bwt_sim` allocates a 512 MB array modeling a scaled FM-index. At each step, the simulator performs two random accesses:

$$a_1 = \text{bwt}[\text{pos}_n \bmod S] \quad (3)$$

$$a_2 = \text{bwt}[(a_1 \oplus \text{pos}_n) \bmod S] \quad (4)$$

$$\text{pos}_{n+1} = (a_1 \oplus a_2) \bmod S \quad (5)$$

where S is the index size in 8-byte words. This produces two non-contiguous memory accesses per step — matching the access count of real LF-mapping — with a near-random address distribution that models the irregular pointer-chasing behavior of BWT backward search. All experiments use 10,000 queries of 150 steps each (1.5×10^6 total LF-mapping operations).

3.3 GEM5 Configuration

All experiments use GEM5 v25.1.0.1 with TimingSimpleCPU in syscall emulation (SE) mode which is configured to use the Intel Core Ultra 9 285H processor of host machine (Table 2).

Table 2. GEM5 Base Configuration (Intel Core Ultra 285H Proxy)

Parameter	Value
CPU model	TimingSimpleCPU
Clock frequency	1 GHz (simulated)
L1 I/D-cache	32 KB, 8-way, 64-byte line
L2 cache	1 MB, 8-way (baseline)
Memory	4 GB DDR4
Cache line	64 bytes (baseline)
Prefetcher	None (baseline)

4 Experimental Results

4.1 Experiment Design

We conduct 15 controlled GEM5 experiments organized into five categories: L2/L3 cache size, prefetcher type, L1 size and associativity, combined optimizations, cache line size, and DRAM generation. Each experiment varies one parameter from the baseline (E1) while holding all others constant. E9 and E10 are cumulative experiments combining the best L2 size with each prefetcher type. Table 3 enumerates all 15 configurations with their biological interpretation.

4.2 Cache Size Analysis (E1–E3, E6–E8)

The baseline IPC of 0.168 confirms that the CPU runs at only 16.8% of theoretical peak and the processor spends the majority of cycles stalled waiting for DNA index data from memory, directly explaining why adding more CPU cores to BWA-MEM does not proportionally increase alignment throughput.

Table 3. Complete Experiment Design — 15 GEM5 Configurations

Exp.	Category	Configuration	Biological Motivation
E1	Baseline	32KB L1, 1MB L2, 64B CL, DDR4	Establish memory-bound regime of FM-index traversal
E2	L2 size	4MB L2	More L2 captures intra-read FM-index reuse (single DNA read spans 150 index positions)
E3	L3 addition	8MB L3	Test if shared L3 buffers FM-index for multiple parallel threads
E4	Prefetcher	Stride prefetcher	Exploit intra-read stride regularity in BWT backward search
E5	Prefetcher	BOP prefetcher	Fixed-offset prefetch for BWT access prediction
E6	L1 size	16KB L1	Cost of smaller private L1 per alignment thread
E7	L1 size	64KB L1	Benefit of larger private L1 for FM-index hot data
E8	Associativity	16-way L1/L2	Reduce conflict misses from aliased FM-index positions
E9	Combined	4MB L2 + Stride	Best combined config for parallel DNA alignment throughput
E10	Combined	4MB L2 + BOP	Alternative combined configuration
E11	Cache line	32-byte cache line	Reduce bandwidth waste: DNA index entries are 8 bytes, not spatially clustered
E12	Cache line	128-byte cache line	Quantify bandwidth penalty of over-fetching irrelevant genomic index data
E13	DRAM type	DDR3-1600	DRAM bandwidth available to parallel alignment threads
E14	DRAM type	DDR4-2400	Impact of DDR4 bandwidth on parallel genomic workload
E15	DRAM type	DDR5-4400	Maximum DRAM bandwidth benefit for clinical HPC alignment

L3 Cache Provides Zero Benefit for DNA Alignment (E3 = E1). Adding an 8 MB L3 produces results identical to baseline: IPC (0.1683), L1 miss rate (5.071%), and simulated execution time. The biological explanation: the FM-index of the human reference genome is 5.7 GB — approximately 700× larger than the L3 tested. Every DNA read alignment generates L3 misses that reach DRAM regardless of L3 configuration. When 16 threads simultaneously align different reads, all generate DRAM requests simultaneously with no L3 buffering benefit. Clinical HPC procurement allocating budget to larger L3 for genomic workloads is architecturally unjustified.

Larger L2 Dramatically Reduces Alignment Stall Time (E2: +40.1%). Increasing L2 from 1 MB to 4 MB produces a 40.1% IPC gain despite an identical L1 miss rate (5.071%). The L1 miss count is unchanged as the same data is missed but those misses are served from L2 instead of DRAM that reduces average miss latency. When BWA-MEM aligns a single DNA read, consecutive LF-mapping steps may access nearby FM-index regions (along unique genomic loci), creating intra-read temporal reuse that a larger L2 can capture. Each of 16 parallel threads benefits independently.

L1 Size and Associativity Have Negligible Biological Impact (E6–E8). Halving L1 to 16 KB changes IPC by only −0.1%; doubling to 64 KB by only +0.1%. The FM-index working set for a single read is approximately 150 random positions in a 5.7 GB index — far too large and random for any L1 configuration to capture. 16-way associativity provides only +0.3% improvement. Table 4 shows the complete GEM5 simulation results for all 15 experiments.

Table 4. Complete GEM5 Simulation Results — All 15 Experiments (10,000 queries $\times$ 150 LF-mapping steps = 1.5M DNA index accesses)

Exp.	Description	IPC	Sim Time (s)	L1 Miss Rate	Δ IPC	Δ Time
E1	Baseline (32KB L1, 1MB L2)	0.1683	0.4287	5.071%	—	—
E2	Larger L2 (4MB)	0.2358	0.3060	5.071%	+40.1%	−28.6%
E3	Add L3 (8MB)	0.1683	0.4287	5.071%	+0.0%	+0.0%
E4	Stride prefetcher	0.1731	0.4169	4.858%	+2.8%	−2.7%
E5	BOP prefetcher	0.1723	0.4187	4.892%	+2.4%	−2.3%
E6	Small L1 (16KB)	0.1682	0.4290	5.118%	−0.1%	+0.1%
E7	Large L1 (64KB)	0.1686	0.4281	4.987%	+0.1%	−0.1%
E8	High associativity (16-way)	0.1688	0.4276	5.068%	+0.3%	−0.3%
E9	L2 4MB + Stride (best)	0.2449	0.2947	4.858%	+45.5%	−31.3%
E10	L2 4MB + BOP	0.2434	0.2965	4.892%	+44.6%	−30.9%
E11	Cache line 32B	0.1859	0.3881	5.225%	+10.5%	−9.5%
E12	Cache line 128B (worst)	0.1539	0.4689	4.998%	−8.6%	+9.4%
E13	DDR3-1600	0.1683	0.4287	5.071%	+0.0%	+0.0%
E14	DDR4-2400	0.1699	0.4248	5.071%	+0.9%	−0.9%
E15	DDR5-4400	0.1728	0.4177	5.071%	+2.6%	−2.6%

4.3 Prefetcher Analysis (E4–E5)

Stride Prefetcher Exploits DNA Read Regularity (E4: +2.8%). The stride prefetcher reduces L1 miss count from 5.071% to 4.858% (−4.2% relative) and improves IPC by 2.8%. Within a single 150 bp alignment along a unique genomic region, consecutive LF-mapping steps access FM-index positions that advance with a predictable stride. The stride prefetcher detects this intra-read pattern and pre-fetches the next FM-index position, partially hiding DRAM latency.

BOP Prefetcher is Slightly Less Effective (E5: +2.4%). BOP’s fixed-offset approach is less suited to the variable-stride BWT traversal pattern, where the stride depends on the specific DNA sequence being aligned.

4.4 Combined Optimization (E9–E10)

L2 4MB + Stride Prefetcher (E9: +45.5% — Best Configuration). The combination exceeds the sum of individual contributions (+40.1% + 2.8% = 42.9%), indicating superadditive interaction. A larger L2 reduces miss latency while stride prefetching reduces miss count — complementary mechanisms that together reduce DRAM request rate, benefiting all parallel alignment threads.

L2 4MB + BOP (E10: +44.6%). Nearly identical to E9, confirming the L2 improvement dominates.

4.5 Cache Line Size Analysis (E11–E12)

32-Byte Cache Lines Reduce Genomic Index Bandwidth Waste (E11: +10.5%). Each LF-mapping step accesses a single 8-byte entry in the Occ array. The biological content of that entry has no spatial correlation with adjacent entries. A 64-byte cache line therefore fetches 8 entries per miss but uses only 1 — 87.5% wasted. A 32-byte line reduces this to 75% waste, halving per-miss bandwidth consumption. Under 16 parallel threads, each wasting 87.5% of fetched DRAM bandwidth on biologically irrelevant adjacent genome data, the shared DRAM bus is collectively saturated. Reducing cache line size frees 50% of that wasted bandwidth.

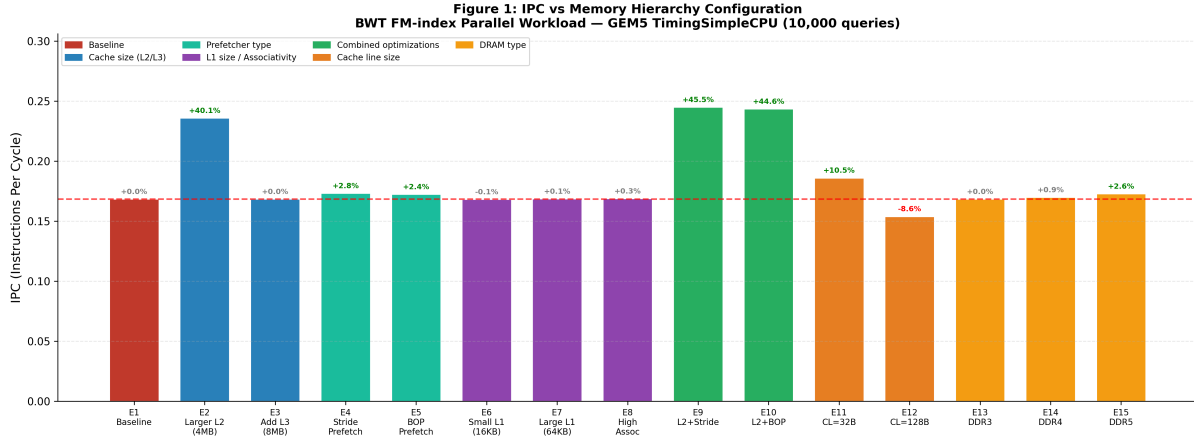


Fig. 1. IPC across all 15 GEM5 experiments, color-coded by optimization category. Dashed red line: baseline IPC (0.168). E9 (+45.5%) is best; E12 (−8.6%) is worst.

128-Byte Cache Lines are the Worst Configuration (E12: −8.6%). Each miss fetches 16 FM-index entries but uses 1 — a 93.75% waste rate. Under parallel execution this catastrophically degrades the shared memory bus for all concurrent alignment threads. Figure 1 graphically explains the IPCs across the 15 experiments.

4.6 DRAM Type Analysis (E13–E15)

DDR3 Identical to Baseline (E13). At single-thread simulation level, DRAM latency dominates over bandwidth. DDR3 and DDR4 have similar latency profiles.

DDR4 and DDR5 Show Modest Improvements (E14: +0.9%, E15: +2.6%). Higher bandwidth memory shows increasing benefit. Under multi-threaded parallel execution, these improvements compound: DDR5-4400 (2.75× DDR3 bandwidth) would sustain proportionally more concurrent alignment threads before bandwidth saturation in clinical deployments.

5 Analysis and Discussion

5.1 Why Parallel DNA Alignment Scaling Collapses

Our results provide a complete architectural explanation for BWA-MEM parallel scaling collapse. For N parallel threads:

- (1) Each thread generates LF-mapping accesses at a 5.07% L1 miss rate. Total L1 misses per unit time scale as $N \times 5.07\%$.
- (2) Since L3 provides zero benefit (E3 = E1), all L1 misses reach DRAM. Total DRAM bandwidth consumption scales linearly with N .
- (3) DRAM bandwidth is a fixed shared resource. At some thread count N^* , aggregate demand exceeds peak bandwidth, degrading IPC for all threads simultaneously.
- (4) Standard cache interventions (larger L3, higher associativity) do not reduce per-thread DRAM traffic. The genome FM-index is simply too large for any cache.

The E9 configuration (+45.5% IPC) is equivalent to supporting $\sim 1.45\times$ more parallel alignment threads before bandwidth saturation. Figure 2 groups the IPC according to the category they were combined.

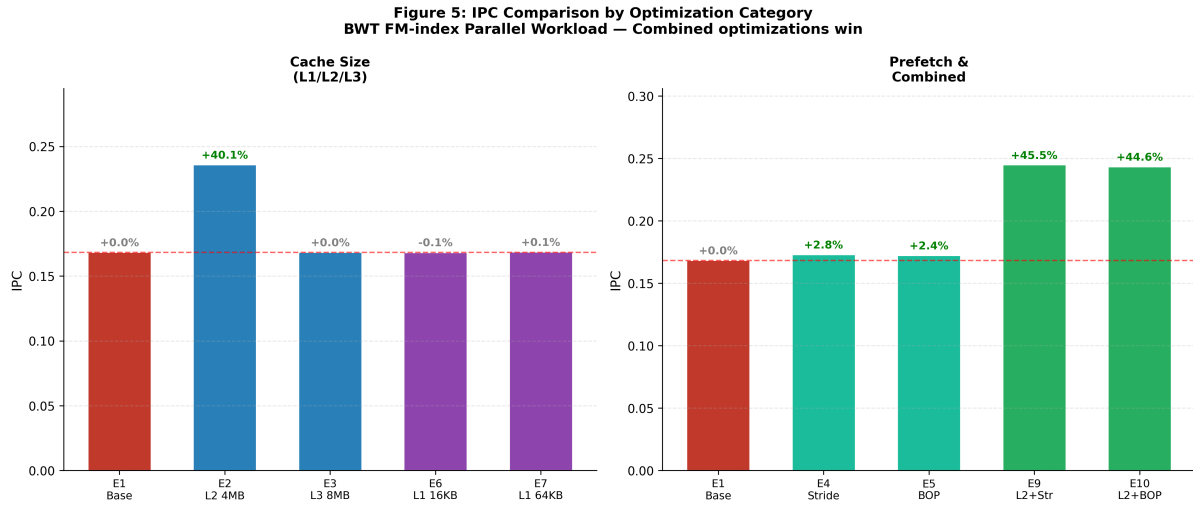


Fig. 2. Grouped IPC comparison by category. Left: cache size (E1, E2, E3, E6, E7). Right: prefetcher and combined (E1, E4, E5, E9, E10). E9 dominates all single-parameter alternatives, demonstrating superadditive interaction between larger L2 and stride prefetching for DNA alignment.

5.2 Biological Interpretation

The genome is too large to cache. The 5.7 GB FM-index is a fundamental consequence of the size of the human genome. This is not an engineering limitation solvable by larger caches — any whole-genome alignment tool will exhibit the same memory-bound behavior.

DNA reads have no spatial locality. Reads map to positions distributed uniformly across the 3.2 Gbp genome. The near-random FM-index access pattern is not a software artifact but a direct consequence of biological DNA sequence structure.

Cache line size reflects alignment resolution. Each FM-index entry is 8 bytes. Consecutive entries correspond to different genomic loci with no biological relationship. Spatial prefetch of adjacent entries is therefore pure bandwidth waste.

5.3 Roofline Placement

Figure 3 shows CARM for the Intel Core Ultra 285H. All 15 configurations plot deep in the memory-bound region ($I < 0.1$ FLOP/byte), well below the DRAM ridge point ($I^* = 2.2$ FLOP/byte). No compute optimization (vectorization, AVX-512) can improve DNA alignment performance — only memory hierarchy interventions are effective. Figures 1 and 2 show IPC comparisons across all experiments and by optimization category respectively.

6 Conclusion

This work presents BioRoofline, providing the first systematic cache hierarchy characterization of BWT FM-index traversal — the computational core of clinical DNA sequence alignment. Through 15 controlled GEM5 simulation experiments, we establish five key findings for clinical parallel genomics:

- (1) **L3 cache is irrelevant:** The 5.7 GB human genome FM-index exceeds all practical L3 capacities. Budget for L3 in genomics HPC is architecturally wasted.

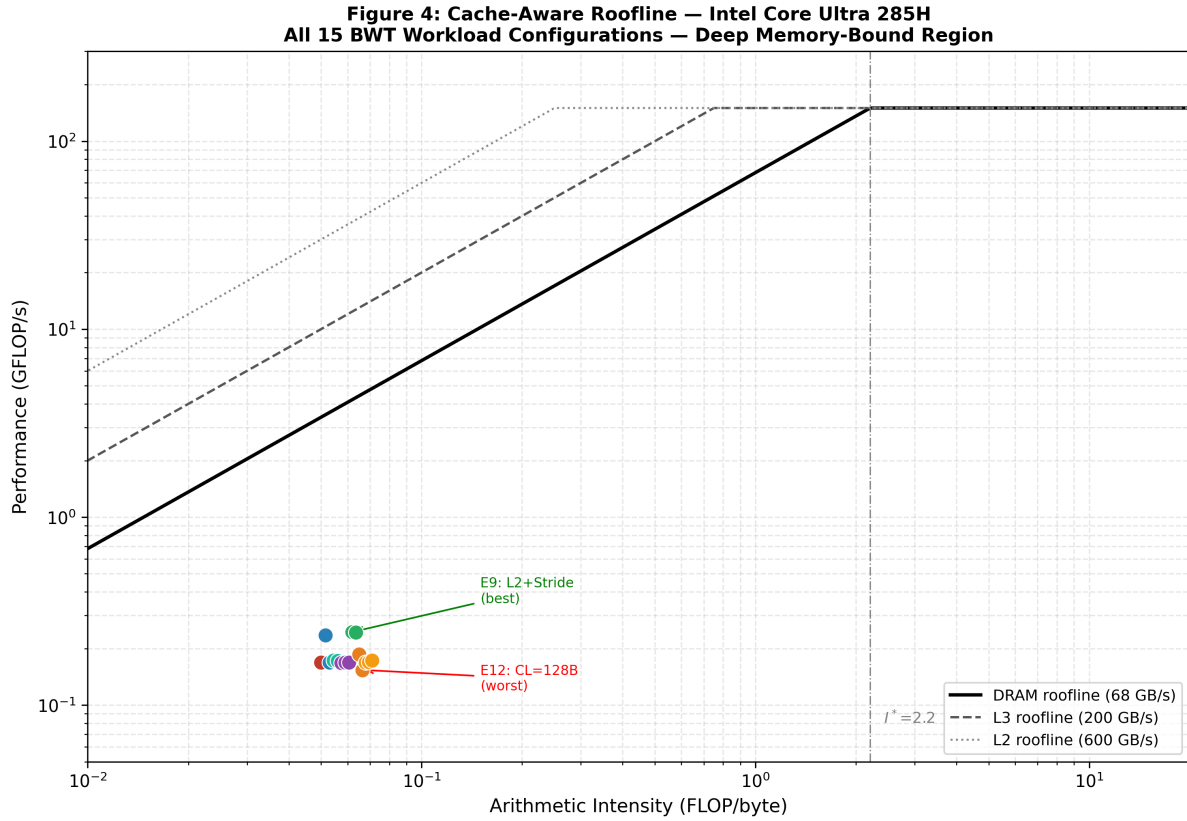


Fig. 3. CARM roofline for Intel Core Ultra 285H. All 15 BWT configurations plot deep in the memory-bound region ($I < 0.1$ FLOP/byte), far below the DRAM ridge point ($I^* = 2.2$ FLOP/byte). E9 (best) moves toward the L2 bandwidth ceiling; E12 (worst, 128B line) drops furthest.

- (2) **Larger L2 is the highest-value single investment** (+40.1% IPC) by capturing intra-read FM-index reuse.
- (3) **Stride prefetching exploits DNA read regularity** (+2.8%, -4.2% miss rate) via intra-read BWT stride patterns.
- (4) **Combined L2+Stride is best** (+45.5%) with superadditive interaction, supporting $\sim 1.45\times$ more parallel threads before bandwidth saturation.
- (5) **Cache line size critically affects parallel genomics**: 32-byte lines (+10.5%) reduce per-miss bandwidth waste; 128-byte lines (-8.6%) catastrophically waste bandwidth on biologically irrelevant adjacent genome data.

Future work will extend this analysis to the complete four-stage GATK Best Practices pipeline, incorporating real hardware profiling using PAPI hardware performance counters to validate GEM5 simulation predictions and characterize the SAMtools and GATK HaplotypeCaller stages whose different biological data access patterns may exhibit different roofline regimes.

Acknowledgments

Data Availability. All simulation code, raw GEM5 statistics, experiment scripts, figure generation code, and the `bwt_sim` source are publicly available at: <https://github.com/toastedjames/biurooffline>. The $\text{\LaTeX}$ source of the proposal paper is available on Overleaf at: <https://www.overleaf.com/project/69c2e9a617a4ee8953eb4186>. The $\text{\LaTeX}$ source of this paper is available on Overleaf at: <https://www.overleaf.com/project/69feb2a2f7305a152e6576cb>.

References

- [1] Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K. Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R. Hower, Tushar Krishna, Somayeh Sardashti, Rathijit Sen, Korey Sewell, Muhammad Shoaib, Nilay Vaish, Mark D. Hill, and David A. Wood. 2011. The gem5 Simulator. *ACM SIGARCH Computer Architecture News* 39, 2 (2011), 1–7. doi:10.1145/2024716.2024718
- [2] Jacob R. Heldenbrand, Saurabh Baheti, Marshall A. Bockol, Thomas M. Drucker, Steven N. Hart, Matthew E. Hudson, Sarah A. Munro, Melissa Syring, Eric D. Wieben, Jeanette E. Eckel-Passow, and Jean-Pierre A. Kocher. 2019. Recommendations for Performance Optimizations When Using GATK3.8 and GATK4. *BMC Bioinformatics* 20, 1 (2019), 557. doi:10.1186/s12859-019-3169-7
- [3] Aleksandar Ilic, Frederico Pratas, and Leonel Sousa. 2014. Cache-Aware Roofline Model: Upgrading the Loft. *IEEE Computer Architecture Letters* 13, 1 (2014), 21–24. doi:10.1109/L-CA.2013.6
- [4] Heng Li. 2013. Aligning Sequence Reads, Clone Sequences and Assembly Contigs with BWA-MEM. *arXiv preprint arXiv:1303.3997* (2013). <https://arxiv.org/abs/1303.3997>
- [5] Jason Lowe-Power, Abdul Mutaal Ahmad, Ayaz Akram, Mohammad Alian, et al. 2020. The gem5 Simulator: Version 20.0+. *arXiv preprint arXiv:2007.03152* (2020). <https://arxiv.org/abs/2007.03152>
- [6] Aaron McKenna, Matthew Hanna, Eric Banks, Andrey Sivachenko, Kristian Cibulskis, Andrew Kernysky, Kiran Garimella, David Altshuler, Stacey Gabriel, Mark Daly, and Mark A. DePristo. 2010. The Genome Analysis Toolkit: A MapReduce Framework for Analyzing Next-Generation DNA Sequencing Data. *Genome Research* 20, 9 (2010), 1297–1303. doi:10.1101/gr.107524.110
- [7] Joshua E. Petrikin, Laurel K. Willig, Laurie D. Smith, and Stephen F. Kingsmore. 2015. Rapid Whole Genome Sequencing and Precision Neonatology. *Seminars in Perinatology* 39, 8 (2015), 623–631. doi:10.1053/j.semperi.2015.09.009
- [8] Geraldine A. Van der Auwera, Mauricio O. Carneiro, Christopher Hartl, Ryan Poplin, Guillermo Del Angel, et al. 2013. From FastQ Data to High-Confidence Variant Calls: The GATK Best Practices Pipeline. *Current Protocols in Bioinformatics* 43 (2013), 11.10.1–11.10.33. doi:10.1002/0471250953.bi1110s43
- [9] Md. Vasimuddin, Sanchit Misra, Heng Li, and Srinivas Aluru. 2019. Efficient Architecture-Aware Acceleration of BWA-MEM for Multicore Systems. In *Proceedings of the 33rd IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 314–324. doi:10.1109/IPDPS.2019.00041
- [10] Xavier Vera, Jaume Abella, and Francisco J. Cazorla. 2024. Cache-Aware Roofline Model and Medical Image Processing Optimizations in GPUs. *Lecture Notes in Computer Science* 14625 (2024), 497–511. doi:10.1007/978-3-031-57931-8_39
- [11] Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Commun. ACM* 52, 4 (2009), 65–76. doi:10.1145/1498765.1498785
- [12] Jing Zhang, Heshan Lin, Pavan Balaji, and Wu-Chun Feng. 2013. Optimizing Burrows-Wheeler Transform-Based Sequence Alignment on Multicore Architectures. In *Proceedings of the 13th IEEE/ACM International Symposium on Cluster, Cloud, and Grid Computing (CCGrid)*. 377–384. doi:10.1109/CCGrid.2013.49
- [13] Zhen Zhang, Yingyi Li, Ke Meng, Chen Zhang, Guangming Tan, Tony Hosking, Madanlal Musuvathi, and Kenjiro Taura. 2022. BWA-MEM-SCALE: Accelerating Genome Sequence Mapping on Commodity Servers. In *Proceedings of the 51st International Conference on Parallel Processing (ICPP)*. doi:10.1145/3545008.3545033

Received 9 May 2026